

Device Management System

A QR-based asset lifecycle platform. Generate labels in bulk, scan to map, track warranties, and route issue tickets straight to the people who fix them.

3

ROLES

42

API ENDPOINTS

9

DB TABLES

21

REPORT EXPORTS

~10.5k

LINES OF CODE

React 18 · Tailwind CSS · Go 1.26 (Gin) · PostgreSQL 18 · JWT + RBAC
PSI Tech · July 2026

1 · What This Project Is

Every physical device in an organisation gets a printed QR sticker. Scan it with any phone and the device's full record opens instantly — specs, photos, videos, manuals, warranty, service history. If it's broken, raise a ticket in two taps and the admin has an email before you've walked away.

The problem it solves

Most organisations track assets in a spreadsheet that nobody updates, or in a system nobody opens because it lives on a desktop in another building. The information exists but is never *at the device*, which is exactly where it's needed — when a machine fails at 2am, or an auditor asks who holds asset AST-1042, or a warranty quietly lapses on 400 laptops at once.

This system puts the record on the device itself. The sticker *is* the index. No app to install, no login to browse, no training required — a phone camera is the entire client.

Who it's for

IT companies, factories, hospitals, educational institutions, and manufacturing organisations — anywhere physical assets are assigned to people, carry warranties, and break.

At a glance

1–5000 QR PER BATCH	28 LABELS PER A4 PAGE	7 REPORT TYPES
3 EXPORT FORMATS	13 UI SCREENS	9 AUDIT ACTION TYPES

2 · Technology Stack

LAYER	TECHNOLOGY	WHY THIS CHOICE
Frontend	React 18 + Vite + Tailwind CSS	Vite gives instant hot reload; Tailwind keeps the design system in one place instead of scattered CSS files.
Backend	Go 1.26 with Gin	Compiles to a single binary with no runtime to install. Handles thousands of concurrent scans on modest hardware.
Database	PostgreSQL 18 via GORM	Transactions matter here — a QR must never map to two devices. Postgres gives real constraints and FILTER aggregates for the dashboard.
Auth	JWT (HS256) + bcrypt	Stateless tokens mean no session store; bcrypt for password hashing.
Charts	Recharts	Declarative, responsive, and theme-aware without custom D3 work.
QR / PDF	go-qrcode, go-pdf/fpdf	Server-side generation, so labels render identically regardless of the client's browser or printer.
Excel	excelize	Real .xlsx with frozen headers and autofilter, not CSV renamed.
Email	gomail (SMTP, port 465 / implicit TLS)	Direct SMTP — no third-party service, no per-email cost, no vendor lock-in.
QR scanning	html5-qrcode	Handles both live camera and still-image decoding, so desktops without webcams still work.

3 · The QR Lifecycle

This is the core workflow the entire system is built around:

1. **GENERATE** Admin creates a batch → DMS000001, DMS000002, DMS000003 ...
status: **available**

2. **PRINT** Download A4 label sheet PDF (28 per page, cut guides)

3. **STICK** Physically attach the labels to devices

4. **SCAN** Anyone points a phone at the sticker

Is this QR mapped yet?

NO

YES

"This QR is not
assigned to any
device. Please
login as Admin."

DEVICE DETAILS PAGE

Overview · Specs · Images
Videos · Manuals · Service
Warranty

Admin logs in

"Raise a query"

5. **MAP** — fill form,
upload media, save

Ticket saved → email to admin
DMS-2026-000001

status: **mapped** (one device per QR, unless remapped)

Why scanning needs no login

The whole point of a sticker on a wall is that anyone can read it. Forcing a login before showing a device's manual would defeat the purpose. Raising a ticket *does* require login — that's how the reporter's identity gets attached automatically.

4 • Roles & Permissions

CAPABILITY	ADMIN	USER	CLIENT
Scan QR & view device details	✓	✓	✓
View images, videos, manuals	✓	✓	✓
Raise a query	✓	✓	—
View own query status	✓	✓	✓
View all queries	✓	—	✓
Generate QR codes in bulk	✓	—	—
Download & print QR labels	✓	—	—
Map QR → device	✓	—	—
Add / edit / delete devices	✓	—	—
Upload images / videos / manuals	✓	—	—
Change query status	✓	—	—
Manage users	✓	—	—
Export reports	✓	—	—
View audit logs	✓	—	—

The one rule that isn't simply "admin or not"

A

Client

sees *every* ticket, read-only. Because a client cannot raise a query at all, scoping them to "their own tickets" would leave the page permanently empty. They already read every device, so the full ticket list is consistent with that. This grants reading only — changing a status stays admin-only.

5 • Feature Summary

QR Management

- Bulk generate 1–5000 per batch
- Sequential unique IDs (**DMS000001**)
- A4 label sheet PDF, 28 per page
- Single large QR PDF for reprints
- Statuses: available, mapped, inactive, lost, replaced
- Search, filter, batch grouping

Device Records

- 18 catalogued fields per device
- Custom key/value specifications
- Multiple images with a primary cover
- Embedded videos, downloadable PDFs
- Service / maintenance history timeline
- Table and card grid views

Query / Ticketing

- Auto ticket numbers (**DMS-2026-000001**)
- 16 fields auto-filled from device + user
- Low / Medium / High priority
- Optional image or PDF attachment
- Email to admin on submit
- Status emails back to the reporter

Dashboard & Analytics

- 12 live KPI cards
- 9 interactive charts
- Warranty expiry timeline
- Category / brand / department splits
- 14-day scan activity trend
- Role-aware — cards adapt per role

Reports

- 7 report types × 3 formats = 21 exports
- Excel with frozen headers + autofilter
- Landscape PDF with zebra striping
- CSV for downstream systems
- Filtered exports from any list page

UI / UX

- Light & dark mode
- Fully mobile responsive
- Toasts, skeletons, confirm dialogs
- Empty states and 403/404 pages
- Camera scan, image upload, or paste
- Smooth animations throughout

6 · Security

CONTROL	IMPLEMENTATION
Password storage	bcrypt hashed. Never returned by any API — the field is <code>json:"-"</code> at the model level, so it cannot leak by accident.
Authentication	JWT HS256, 12-hour expiry, signed server-side.
Authorisation	RBAC enforced per route group. The role is re-read from the database on every request, so deactivating a user takes effect immediately rather than whenever their token happens to expire.
Rate limiting	Per-IP token bucket. Login is throttled far harder (0.2 req/s, burst 8) than the global ceiling, to blunt credential stuffing.
File uploads	Extension <i>and</i> sniffed content-type must both match — a <code>.png</code> that is actually a script is rejected. Stored under a random UUID filename, never the client-supplied one.
SQL injection	Parameterised queries throughout. <code>ORDER BY</code> columns are whitelisted, since they cannot be parameterised.
Enumeration	Unknown email and wrong password return the identical message, so the login form cannot be used to discover which accounts exist.
Audit trail	Every mutating action logs user, role, action, reference, IP, and timestamp.
Secrets	SMTP and DB credentials live in <code>.env</code> , which is gitignored and never sent to the browser.

7 · Questions & Answers

The questions people actually ask when they first see this system — grouped by who tends to ask them.

GENERAL & BUSINESS

Q What problem does this actually solve?

A Asset information is normally stored far away from the asset. This puts it *on* the asset. Scan the sticker, get the record. It removes the "who owns this, when does its warranty end, where's the manual" scramble entirely — and it makes reporting a fault take seconds instead of an email thread.

Q Who can use it — do people need to install an app?

A No app. Any phone camera opens the device page in a browser. Viewing needs no login at all; only raising a ticket does.

Q What kind of organisations is this for?

A Anywhere physical assets are assigned to people and break: IT companies, factories, hospitals, schools, manufacturing. Anything with a serial number and a warranty fits the model.

Q How much does it cost to run?

A No per-user or per-scan licensing — it's your own code on your own server. Email goes out over your existing SMTP account, so there's no per-email cost either. The real costs are a server, a database, and sticker paper.

QR CODES

Q How are QR codes generated, and are they guaranteed unique?

A Yes, guaranteed. IDs come from a database counter incremented with a single atomic `UPDATE ... RETURNING` statement. Two admins clicking "Generate 500" at the same instant get two separate, non-overlapping blocks — a collision is not merely unlikely, it's impossible by construction.

Q What's actually inside the QR code?

A A URL: `https://yourdomain.com/device/DMS000001`. Nothing else — no data is embedded in the code itself, so the record can change freely without reprinting the sticker.

Q What happens the first time someone scans an unmapped sticker?

A They see: *"This QR is not assigned to any device. Please login as Admin."* If an admin is already logged in, they instead get a direct button to map it. Either way the scan is still logged.

Q Can one QR code be mapped to two devices?

A No. It's enforced at two levels: a check in the mapping handler, and a unique database constraint on `qr_code_id`. Even if two admins submit the mapping form simultaneously, the database rejects the second one.

Q What if a sticker is damaged, lost, or peels off?

A Mark the QR as **lost** or **replaced** — scanning it then reports it's out of use rather than showing the device. Reprint a single large QR PDF for the same code, or unmap and remap to a fresh code.

Q How many QR codes can it handle?

A 5000 per batch, unlimited batches. The ID format holds 999,999 codes before it needs another digit. Inserts are batched 500 at a time, so generating 5000 is a handful of round trips, not 5000.

Q Can I change the domain after printing stickers?

A This is the one genuinely painful mistake. The URL is **baked into the QR image at generation time** — it is not resolved when scanned. Change the domain and every already-printed sticker points at the old address. Set `PUBLIC_BASE_URL` to the real domain *before* generating production codes. Existing codes can be reprinted in the database only if the domain still resolves.

Q Do I need a special printer or label stock?

A Any printer. The sheet is plain A4 with 28 labels and cut guides — use adhesive sticker paper. Print at 100% scale; "fit to page" will shrink the codes and can make them unreadable.

QUERIES & EMAIL

Q Where does a query go — the database or email?

A **Both, deliberately.** The ticket is saved to the database first; the email is a *notification*, not the storage. If email were the only copy, one mail outage would lose tickets permanently, and there'd be no queries list, no status tracking, and no reports.

Q What if the mail server is slow or down — do we lose the ticket?

A No. The ticket commits to the database, *then* the email is attempted on a background goroutine. A dead SMTP server produces a log line, not a failed submission. The user always gets their ticket number.

Q What's in the admin email?

A Everything: ticket number, device number, QR number, device name, brand, model, serial, company, project, department, assigned employee, location, reporter name, employee ID, email, priority, title, description, and timestamp. Any attachment comes through attached. `Reply-To` is set to the reporter, so replying from the inbox reaches them directly.

Q How do I change who receives the notifications?

A Change `ADMIN_EMAIL` in `backend/.env` and restart the backend. It's independent of `SMTP_USERNAME` (the account that *sends*) — the two are commonly confused. Note `.env` is read once at startup, so a restart is required.

Q Why doesn't the user type the device details on the query form?

A Because they'd get them wrong, and because we already know them. The user enters only title, description, and priority. All 16 device and reporter fields are read server-side from the database — the form *displays* them so the user can see what's attached, but never sends them. This also means the client cannot forge which device a ticket is against.

Q Why does a ticket store copies of the device fields instead of just linking?

A Deliberate denormalisation. A ticket is a historical record. If a laptop is reassigned from Rahul to Priya next month, a ticket Rahul raised must still say Rahul — otherwise history silently rewrites itself and audits become meaningless.

Q Are ticket numbers sequential? Can two collide?

A Sequential per year — `DMS-2026-000001`. Same atomic counter as QR IDs, and the number is allocated inside the same transaction as the insert, so a failed insert never burns a number.

Q Does the reporter get told when their ticket is resolved?

A Yes — every status change emails them, including the admin's remarks. That send is best-effort too: an undeliverable reporter address is logged and never blocks the admin's update.

TECHNICAL & ARCHITECTURE

Q Why Go instead of Node or Python?

A It compiles to one binary with no runtime to install or keep patched, starts instantly, and handles high concurrent scan traffic on small hardware. For a system whose load is thousands of short-lived scan requests, that fits well.

Q Why PostgreSQL rather than MySQL or MongoDB?

A The core invariants are relational and need real transactions — one QR maps to exactly one device; a ticket number and its row must be allocated atomically. Postgres also provides `COUNT(*) FILTER (WHERE ...)`, which lets each dashboard card come from one query rather than many.

Q Is the API secure if someone bypasses the UI?

A Yes — the UI is not a security boundary. It hides what a role can't do purely to avoid dead ends; every rule is enforced server-side per route group. Calling the API directly with a User token still gets a 403 on admin routes. This is tested per role.

Q What happens if I deactivate a user mid-session?

A They lose access on their very next request. The role and active flag are re-read from the database on every call rather than trusted from the token — so you never have to wait out a token's 12-hour expiry.

Q What happens when a device is deleted?

A The device, its media rows, and its service history are removed in one transaction, and its QR code returns to **available** for reuse. The physical files are deleted only *after* the transaction commits — so a rolled-back delete never leaves the database pointing at missing files. Tickets raised against it survive, because they hold their own snapshot.

Q Can uploaded files be malicious?

A Both the extension and the sniffed content-type must match before anything is written — a script renamed `.png` is rejected. Files are stored under random UUID names, so a client-supplied filename can't traverse paths or overwrite anything.

Q Is there an audit trail?

A Yes — QR generated, QR mapped, device created/updated/deleted, user login, QR scanned, query submitted, status changed, report exported. Each row records user, role, action, reference, IP, and timestamp, is filterable by date and action, and exports to Excel.

Q Does it work on mobile?

A Fully responsive throughout — and mobile is the primary scanning surface, so the device page is designed phone-first.

Q What if the scanning device has no camera?

A Three ways in: live camera, upload a photo or paste a screenshot (`Ctrl+V`) which decodes the image with no camera permission at all, or type the code by hand.

Q How does it scale?

A Every list endpoint is paginated and capped at 200 rows per page. Filterable and searched columns are indexed. Report exports stream as a download rather than buffering in the browser. The realistic first bottleneck is video storage on disk, not the database.

OPERATIONS & DEPLOYMENT

Q What do I need to run it?

A PostgreSQL, and the compiled Go binary plus the built frontend. In development it's Go 1.26 and Node. Tables and demo accounts are created automatically on first run — there's no separate migration step.

Q How do I back it up?

A Two things: `pg_dump` the database, and copy the `uploads/` directory. The database holds every record; `uploads/` holds the images, videos, and manuals. Backing up only one gives you a broken restore.

Q What must change before going live?

- A**
1. Set a long random `JWT_SECRET` (64+ chars).
 2. Change all three seeded demo passwords.
 3. Set `APP_ENV=production`.
 4. Point `PUBLIC_BASE_URL` at the real domain **before generating any QR codes**.
 5. Use a dedicated database role, not `postgres`.
 6. Put the API behind HTTPS.
 7. Add auth to `/uploads` if manuals are confidential.

Q Can multiple clients be isolated from each other?

- A** Not yet — today a Client sees every ticket and device. If you onboard multiple external clients who must not see each other's data, scope queries and devices by the client's `company` field. Worth doing *before* real client accounts exist, not after.

Q What about WhatsApp notifications?

- A** Built and wired for the Meta Cloud API, currently disabled. The message builder is done. Set `WHATSAPP_ENABLED=true` plus the token and phone ID to switch it on — no code change needed.

Q What are the known limitations?

- A**
- No multi-tenant isolation between clients (see above).
 - No password-reset email flow — an admin resets passwords.
 - Uploads go to local disk, not S3 — fine on one server, needs shared storage to run several.
 - WhatsApp untested against a live account.
 - No offline scanning; the device page needs a connection.

8 · Project Structure

Database tables

TABLE	PURPOSE
users	Accounts, roles, employee details
qr_codes	Every generated code, its status, batch, and scan count
devices	The asset record — one per mapped QR
media	Images, videos, manuals attached to a device
service_records	Maintenance and repair history
queries	Tickets, with a snapshot of device + reporter
scans	Every QR scan — time, IP, mapped or not
audit_logs	Who did what, when, from which IP
counters	Atomic sequences for asset IDs and ticket numbers

Key endpoints

METHOD	ENDPOINT	ACCESS	PURPOSE
POST	/api/auth/login	public	Get a JWT
GET	/api/scan/:assetId	public	QR scan — mapped or not
POST	/api/qr/generate	admin	Bulk generate (1–5000)
POST	/api/qr/print	admin	A4 label sheet PDF
POST	/api/qr/:assetId/map	admin	Map QR → device
POST	/api/queries	admin + user	Raise a query → email
PATCH	/api/queries/:id/status	admin	Change ticket status
GET	/api/reports/:type	admin	?format=excel pdf csv

9 • Notable Design Decisions

Five choices worth understanding, because each one has a non-obvious reason:

1 · Atomic counters, not `MAX(id)+1`

Asset IDs and ticket numbers come from a single `UPDATE counters SET value = value + n ... RETURNING`. The naive read-then-write approach silently produces duplicates the moment two admins act at once — and that is exactly the kind of bug that only shows up in production.

2 · Tickets snapshot their device

A query copies the device and reporter fields at submit time rather than joining. It costs storage but keeps history truthful: reassigning a device later must not rewrite who reported what.

3 · Notifications never block the request

Email is sent on a background goroutine *after* the ticket commits. A slow SMTP server must never make a user think their query failed.

4 · Public scan, authenticated action

Reading a device needs no login — that's the point of a sticker. Raising a ticket does, which is how the reporter's identity is attached automatically and cannot be forged.

5 · Files deleted after commit, never before

Deleting a device removes its rows in a transaction, and only touches the disk once that transaction has committed. Deleting files first would leave the database pointing at nothing if the transaction rolled back.